

Documentacion Final - Proyecto 4

VeriLang en Kotlin: ejecucion del lenguaje Rascal desde una aplicacion Kotlin

Campo	Informacion
Curso	ISIS-2111 - Elementos Esenciales de Lenguajes de Programacion
Estudiante	Ronny Esteban Lopez Martinez
Codigo	202415741
Proyecto	Proyecto 4 - VeriLang ejecutado desde Kotlin
Raiz del proyecto	proyecto_rascal
Contenido de la entrega	Proyecto Rascal, aplicacion Kotlin, jar de ejecucion Rascal, casos de prueba y documento explicativo final

Correcciones del Proyecto 3 incorporadas en la version final

Esta seccion se incluye al comienzo del documento porque la version final de VeriLang no solo migra la ejecucion del lenguaje a Kotlin, sino que tambien conserva y hace visibles las correcciones funcionales asociadas a los criterios evaluados en el Proyecto 3. La evidencia de referencia corresponde a la rubrica y retroalimentacion recibida para tres aspectos: actualizacion de gramatica y anotaciones de tipo, correspondencia de tipos y regla de existencia de elementos en estructuras de datos. En la implementacion final estos aspectos quedan integrados en la gramatica, el AST, el generador, el checker con TypePal, los casos de prueba .vl, el puente RunnerJson.rsc y la aplicacion Kotlin.

Criterios	Excellent 5 points	Proficient 4 points	Competent 3 points	Needs improvement 2 points	Unsatisfactory 1 point	Criterion Score
Grammar Update & Type Annotations	The grammar is correctly updated to include type annotations for all required values (Integers, Booleans, Chars, Strings) and user-defined data structures	The grammar is mostly updated, but annotations for one specific type (e.g., Chars or a structure field) might be slightly inaccurate or missing. ✓	Grammar updates for annotations are present but contain significant errors or fail to cover multiple required types and data structures	Grammar updates are attempted but fundamentally flawed or only cover trivial type annotations.	The grammar is not updated to include type annotations	4 / 5
Criterion Feedback No se incluye soporte para los tipos definidos por el usuario						
Type Correspondence Checking	The type system correctly checks that the types used in the program correspond precisely to their definitions, ensuring type safety across the entire program	Type checking is functional and correct for standard cases but may fail on specific, complex scenarios or nested type interactions ✓	Type checking is implemented but fails frequently on common type misuse, indicating a significant flaw in the type rules	Type checking is attempted but only covers a small subset of type correspondence rules or is highly unreliable.	Type correspondence checking is missing or non-functional	4 / 5
Criterion Feedback No se revisan los tipos que están dentro de definiciones anidadas.						
Data Structure Element Existence Rule	A new, correct rule is defined and implemented to verify the existence of all elements used within a data structure definition. This rule is reliably enforced.	The element existence rule is defined and works for most cases, but may fail on specific edge cases ✓	The element existence rule is implemented but fails to catch non-existent elements in common scenarios, or the rule definition is fundamentally flawed	The rule is attempted but only checks for trivial data structure element existence or is entirely incorrect	The new rule to verify element existence is missing or not implemented	4 / 5
Total						
12 / 15						

Figura 1. Rubrica y retroalimentacion usadas como referencia funcional: anotaciones de tipo, correspondencia de tipos y existencia de elementos en estructuras.

1.1 Lectura tecnica de los criterios corregidos

La rubrica de referencia exige que la gramatica incluya anotaciones de tipo para los valores requeridos, incluyendo enteros, booleanos, caracteres, cadenas y estructuras definidas por el usuario. Tambien exige que el sistema de tipos compruebe que los tipos usados en el programa correspondan precisamente con sus definiciones y que exista una regla confiable para verificar los elementos usados dentro de estructuras de datos. La retroalimentacion concreta senalaba tres puntos funcionales que debian quedar cubiertos: soporte para tipos definidos por usuario, revision de tipos dentro de definiciones anidadas y fortalecimiento de la regla de existencia de elementos.

Elemento observado en la evaluacion	Interpretacion tecnica aplicada en la solucion final
No se incluye soporte para los tipos definidos por el usuario	VeriLang permite que nombres declarados con defspace sean usados como dominios de variables, operadores, estructuras tipadas y cuantificadores. Estos nombres entran al sistema de tipos como userType y se resuelven contra declaraciones reales de espacios.
No se revisan los tipos que estan dentro de definiciones anidadas	El checker entra a expresiones compuestas, operadores infijos, reglas, invocaciones y cuantificadores. Si una expresion anidada depende de una estructura tipada, el tipo de la variable ligada corresponde al tipo de elemento de esa estructura.
La regla de existencia funciona para la mayoría de casos, pero puede fallar en casos especiales	La regla se aplica a cada uso de un tipo definido por usuario: variables, firmas de operadores, estructuras tipadas y estructuras usadas por cuantificadores. Si el tipo no existe como defspace, se produce un error semantico controlado.

1.2 Correccion 1: anotaciones de tipo y tipos definidos por usuario

La primera correccion incorporada es el soporte completo para tipos definidos por el usuario. VeriLang conserva los tipos primitivos int, bool, real, string y char, y ademas permite que cualquier espacio definido con defspace pueda actuar como dominio de variables, parametros de operadores y estructuras. La gramatica concreta reconoce estos dominios mediante identificadores, el AST conserva la informacion de estructuras tipadas y el generador vuelve a producir la notacion tipada del lenguaje. De esta manera, una declaracion como defspace Group : Person end tiene significado semantico: Group es una estructura cuyos elementos tienen tipo Person.

```

syntax Domain
= boolDomain: 'bool'
| intDomain: 'int'
| realDomain: 'real'
| stringDomain: 'string'
| charDomain: 'char'
| nameDomain: ID domainName
;

syntax Space
= space: 'defspace' ID spaceName 'end'
| typedSpace: 'defspace' ID spaceName ':' Domain elementType 'end'
| subSpace: 'defspace' ID subSpace '<' ID superSpace 'end'
| typedSubspace: 'defspace' ID subSpace '<' ID superSpace ':' Domain elementType 'end'
;

```

La correccion se evidencia en los programas de prueba incluidos en instance/. El archivo instance/spec_types.vl declara tipos definidos por usuario y los usa en variables y operadores. Por ejemplo, Person y Group son espacios del lenguaje; p y q tienen tipo Person; g tiene tipo Group; belongsTo recibe un Person y un Group y retorna bool. Este caso ya no es solamente una prueba de sintaxis, sino una prueba de integracion entre gramatica, AST, generador y checker.

```
defspace Person end
defspace Group : Person end

defvar
  p : Person
  q : Person
  g : Group
end

defoperator samePerson : Person -> Person -> bool end
defoperator belongsTo : Person -> Group -> bool end
```

1.3 Correccion 2: correspondencia de tipos en expresiones y definiciones anidadas

La segunda correccion consiste en validar la correspondencia de tipos mas alla de casos simples. El checker implementado con TypePal no se limita a revisar que existan nombres, sino que calcula y compara los tipos de expresiones completas. Esto incluye igualdad entre enteros, reales, cadenas, caracteres y tipos definidos por usuario; operadores logicos sobre booleanos; operadores personalizados declarados con firmas; reglas; invocaciones; y expresiones cuantificadas. La parte central de la correccion esta en las expresiones anidadas: cuando una expresion contiene un cuantificador sobre una estructura tipada, la variable ligada toma el tipo del elemento declarado en la estructura, no simplemente el nombre de la estructura.

```
defspace Person end
defspace Group : Person end

defoperator belongsTo : Person -> Group -> bool end

defexpression (forall member in Group . member belongsTo g) end
```

En este ejemplo, Group esta declarado como Group : Person. Por tanto, member dentro del cuantificador debe ser tratado como Person. La expresion member belongsTo g solo es correcta si el lado izquierdo del operador belongsTo corresponde a Person y el lado derecho corresponde a Group. La version final conserva esta informacion en un mapa semantico de tipos de elementos de estructuras, lo cual permite que el checker revise correctamente el tipo dentro de la definicion anidada.

```
map[str, AType] typedSpaceElementTypes = ();

AType quantifiedVariableType(str domainName) {
  if (domainName in typedSpaceElementTypes) {
    return typedSpaceElementTypes[domainName];
  }
  return userType(domainName);
}
```

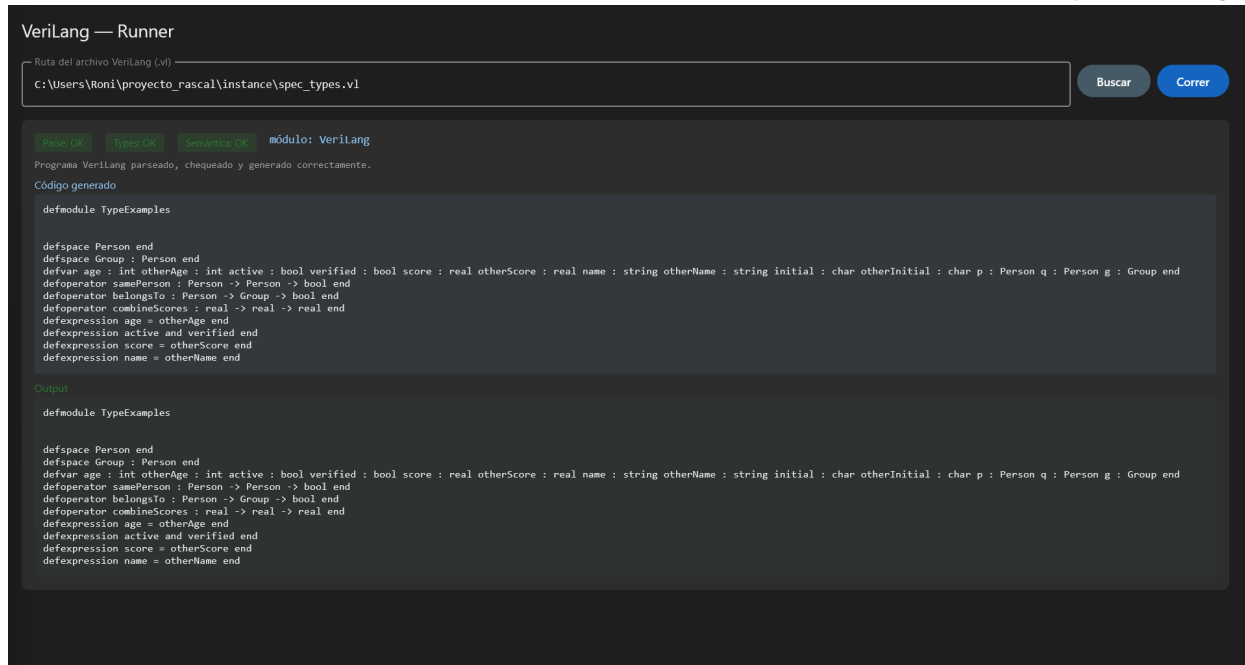


Figura 2. Evidencia de ejecución válida desde Kotlin: `spec_types.vl` usa tipos definidos por usuario y cuantificador sobre `Group : Person`; la interfaz reporta `Parse OK`, `Types OK` y `Semantics OK`.

1.4 Corrección 3: regla de existencia de elementos en estructuras tipadas

La tercera corrección es la regla de existencia de elementos en estructuras de datos. Una estructura tipada no puede declarar un tipo de elemento que no exista. Por ejemplo, `defspace Group : Person end` solo es válido si `Person` fue declarado antes como espacio del lenguaje. Esta regla se implementa usando los roles de identificadores del checker: variables, operadores y espacios se distinguen mediante roles, y los dominios definidos por usuario se registran como usos que deben resolverse contra declaraciones de espacios.

```
data IdRole
= variableId()
| operatorId()
| spaceId()
;

void useDomainIfUserDefined(Domain d, Collector c) {
    if (domainToType(d) is userType) {
        c.use(d, {spaceId()});
    }
}
```

El comportamiento esperado se observa en los casos inválidos. Si se usa `Person` como tipo de elemento sin declararlo como `defspace`, `TypePal` reporta `Undefined space `Person``. La aplicación Kotlin conserva exactamente ese comportamiento porque no reemplaza el sistema de tipos: invoca `RunnerJson.rsc`, que a su vez usa `Checker.rsc` y `TypePal`. Por eso el caso inválido mantiene parseo correcto, pero falla en tipos y semántica.

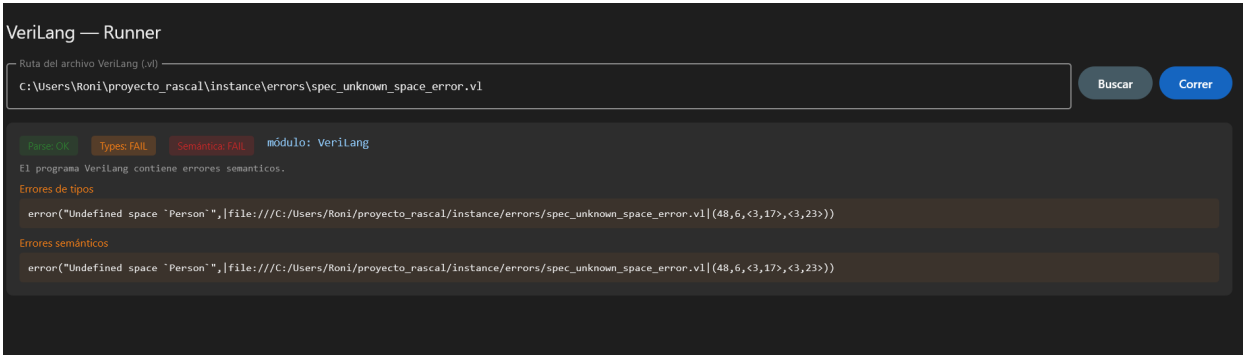


Figura 3. Evidencia de ejecucion invalida desde Kotlin: la estructura usa Person sin declararlo; la interfaz reporta Types FAIL, Semantica FAIL y Undefined space `Person`.

1.5 Trazabilidad entre retroalimentacion, archivos y pruebas

Aspecto corregido	Archivos principales	Pruebas asociadas	Resultado observado
Tipos definidos por usuario	Syntax.rsc, AST.rsc, Generator.rsc, Checker.rsc	instance/spec_types.vl, instance/spec_typed_space_good.vl	Los programas validos con Person, Group, Set y Element ejecutan con Types OK.
Tipos dentro de definiciones anidadas	Checker.rsc, RunnerJson.rsc	instance/spec_types.vl, instance/spec_typed_space_quantifier_good.vl, instance/errors/spec_typed_space_quantifier_type_error.vl	Los cuantificadores sobre estructuras tipadas asignan a la variable ligada el tipo de elemento correspondiente.
Existencia de elementos en estructuras	Checker.rsc, TypePal, Main.rsc, RunnerJson.rsc	instance/errors/spec_unknown_space_error.vl, instance/errors/spec_typed_space_unknown_element_error.vl	El sistema reporta Undefined space cuando se usa un tipo de usuario no declarado.
Conservacion en Kotlin	LangService.kt, RunResult.kt, MainWindow.kt	Ejecucion grafica de spec_types.vl y spec_unknown_space_error.vl	La interfaz muestra estados OK para el valido y FAIL controlado para el invalido.

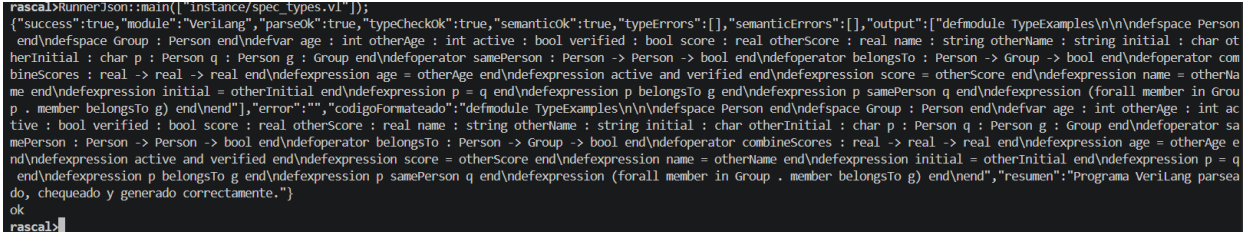


Figura 4. Evidencia de RunnerJson en Rascal: el puente produce JSON con success true para un programa valido.

1.6 Relacion entre las correcciones y el Proyecto 4

```
Kotlin UI
-> LangService.kt
    -> rascal-shell-stable.jar
        -> RunnerJson.rsc
            -> Parser.rsc
            -> Checker.rsc + TypePal
            -> Generator.rsc
        <- JSON
    <- RunResult.kt
-> MainWindow.kt muestra estados y errores
```

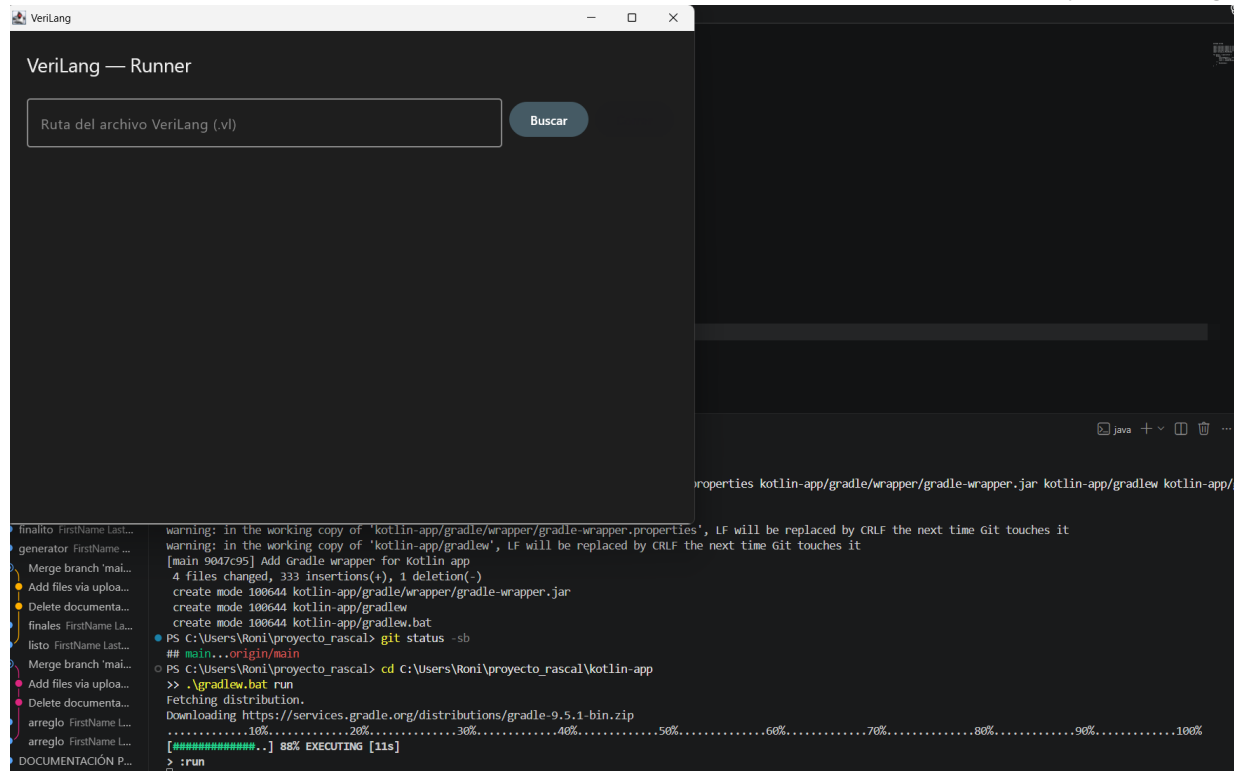


Figura 6. Evidencia de ejecucion portable con Gradle Wrapper: `.\gradlew.bat run` abre la aplicacion VeriLang.

Con esta trazabilidad, la version final cubre explícitamente los criterios funcionales del lenguaje y ademas los expone desde Kotlin. Las capturas incluidas muestran que el caso valido ejecuta con parseo, tipos y semantica correctos, mientras que el caso invalido conserva el parseo pero reporta el error semantico esperado. La seccion siguiente presenta el resto de la arquitectura del Proyecto 4.

Resumen ejecutivo

Este documento presenta la version final del Proyecto 4 de VeriLang. La solucion reutiliza la implementacion del Proyecto 3 en Rascal, que contiene parser, generador y sistema de tipos con TypePal, y la integra con una aplicacion Kotlin basada en el helper entregado para el proyecto. La aplicacion permite seleccionar archivos `.vl`, ejecutar el flujo del lenguaje, visualizar el resultado generado y reportar errores semanticos desde una interfaz grafica de escritorio.

La integracion se realizo mediante un modulo puente llamado `RunnerJson.rsc`. Este modulo invoca el parser, el checker y el generador de VeriLang, y devuelve un objeto JSON consumido por Kotlin. La aplicacion Kotlin ejecuta Rascal como subprocesso, interpreta el JSON con `kotlinx.serialization` y presenta el estado de parseo, chequeo de tipos, validacion semantica, salida generada y errores encontrados.

Tabla de contenido

1. Introduccion y objetivo
2. Requisitos abordados por la solucion
3. Estructura final del proyecto
4. Arquitectura de integracion Rascal-Kotlin
5. Componentes Rascal reutilizados
6. Modulo RunnerJson.rsc
7. Aplicacion Kotlin
8. Flujo de ejecucion
9. Configuracion y dependencias
10. Casos de prueba y evidencias
11. Cobertura de criterios funcionales del lenguaje
12. Como ejecutar el proyecto
13. Contenido de entrega
14. Conclusiones

1. Introduccion y objetivo

El objetivo del Proyecto 4 es tomar la implementacion existente de VeriLang en Rascal y ejecutarla desde el entorno de Kotlin. La solucion conserva la funcionalidad central del Proyecto 3 y agrega una capa de ejecucion grafica en Kotlin que actua como runtime e interfaz de usuario para el lenguaje.

El enunciado del proyecto establece que la implementacion previa de VeriLang debe reutilizarse, que el nuevo objetivo es Kotlin y que el resultado debe ser funcionalmente equivalente al Proyecto 3. Por esta razon, la implementacion final mantiene los modulos Rascal originales y agrega una comunicacion controlada entre Kotlin y Rascal.



ISIS-2111 PLE
Project 4
Fecha: May 8, 2026

Task 1. For the fourth part of the project, we will explore the capabilities of Rascal as a language workbench and the power of the definition of languages using formal grammars. To do this, your task is to reuse the implementation of VeriLang from the previous project, but now we will target a different language (i.e., not Java), Kotlin.

Your project should be functionally equivalent to project 3, but now run from the Kotlin runtime environment.

In addition to the project you will find a Kotlin helper file, that will help you in deploying your VeriLang to Kotlin.

Hand in a .zip file with the solution and document.

1

Figura 1. Enunciado del Proyecto 4: reutilizar VeriLang, apuntar a Kotlin y entregar solucion con documento.

2. Requisitos abordados por la solucion

Requisito	Implementacion en la solucion final
Reutilizar VeriLang del Proyecto 3	Se conservan Syntax.rsc, AST.rsc, Parser.rsc, Implode.rsc, Generator.rsc, Checker.rsc, Main.rsc y Plugin.rsc.
Ejecutar desde Kotlin	Se integra una aplicacion de escritorio en kotlin-app/ usando Compose Desktop.
Mantener equivalencia funcional	Desde Kotlin se puede parsear, chequear, reportar errores y mostrar la salida generada por VeriLang.
Usar el helper Kotlin	El helper se adapta mediante Main.kt, MainWindow.kt, LangService.kt y RunResult.kt.
Entregar solucion ejecutable	Se incluye rascal-shell-stable.jar y Gradle Wrapper para ejecutar la aplicacion sin depender de Gradle global.

3. Estructura final del proyecto

La raíz del proyecto contiene la implementacion Rascal original, la aplicacion Kotlin, los archivos de prueba y los archivos de configuracion necesarios para ejecutar la entrega final.

```

proyecto_rascal/
  META-INF/
    RASCAL.MF
  src/main/rascal/
    Syntax.rsc
    AST.rsc
    Parser.rsc
    Implode.rsc
    Generator.rsc
    Checker.rsc
    Main.rsc
    Plugin.rsc
    RunnerJson.rsc
  instance/
    spec_types.vl
    spec_typed_space_good.vl
    spec_typed_space_quantifier_good.vl
    errors/
      spec_unknown_space_error.vl
      spec_typed_space_quantifier_type_error.vl
    ...
  kotlin-app/
    build.gradle.kts
    settings.gradle.kts
    gradlew
    gradlew.bat
    gradle/wrapper/
  src/main/kotlin/milang/
    Main.kt
    model/RunResult.kt
    service/LangService.kt
    ui/MainWindow.kt
  pom.xml
  rascal-shell-stable.jar
  
```

Archivo o carpeta	Funcion
src/main/rascal/	Contiene la implementacion principal de VeriLang en Rascal.
RunnerJson.rsc	Puente de ejecucion entre Kotlin y los modulos Rascal.
kotlin-app/	Aplicacion Kotlin/Compose que permite seleccionar y ejecutar archivos .vl.
rascal-shell-stable.jar	Jar usado por Kotlin para abrir un proceso Rascal.
instance/	Programas VeriLang validos e invalidos usados en las pruebas.
pom.xml	Dependencias Maven de Rascal y TypePal.
.gitignore	Excluye artefactos generados como target/, build/, bin/, .gradle/ y archivos .lock.

4. Arquitectura de integracion Rascal-Kotlin

La arquitectura mantiene separadas las responsabilidades del lenguaje y de la interfaz. Rascal conserva la definicion y validacion del lenguaje, mientras que Kotlin actua como runtime grafico y consumidor del resultado JSON.

```

Usuario selecciona archivo .vl
|
v
MainWindow.kt
|
v
LangService.kt
|
v
rascal-shell-stable.jar
|
v
RunnerJson.rsc
|
+--> Parser.rsc
+--> Checker.rsc + TypePal
+--> Generator.rsc
|
v
JSON de resultado
|
v
RunResult.kt
|
v
Interfaz Kotlin: Parse / Types / Semantica / Output / Errores
  
```

El intercambio entre Rascal y Kotlin se hizo mediante JSON para evitar acoplar directamente la UI a estructuras internas de Rascal. Esta decision permite mostrar los resultados de forma uniforme, tanto en casos correctos como en casos con errores semanticos.

5. Componentes Rascal reutilizados

La base Rascal corresponde a VeriLang del Proyecto 3. Esta base incluye parser, AST, generador y checker con TypePal. El lenguaje reconoce modulos, espacios, variables, operadores, reglas, expresiones, cuantificadores, tipos primitivos y tipos definidos por el usuario.

Componente	Responsabilidad
Syntax.rsc	Define la gramatica concreta de VeriLang.
AST.rsc	Define la sintaxis abstracta usada por generador y checker.
Parser.rsc	Expone parseMainModule para leer archivos .vl.
Implode.rsc	Transforma el arbol concreto en AST.

```
[INFO] pom.xml: Rascal version is 0.42.7
[INFO] pom.xml: Project root is [file:///C:/Users/Roni/projecto_rascal/]
[INFO] pom.xml: Bin folder is [file:///C:/Users/Roni/projecto_rascal/target/classes]
[INFO] pom.xml: Source module path is:
  - [std:///]
  - [mvn://org.rascalml1--typepal--0.8.6/src]
  - [file:///C:/Users/Roni/projecto_rascal/src/main/rascal]
  - [jar+file:///C:/Users/Roni/.vscode/extensions/usethesource.rascalml1-0.13.5/assets/jars/rascal-lsp.jar!library]
[INFO] pom.xml: Library module (and classes) path is:
  - [file:///C:/Users/Roni/.vscode/extensions/usethesource.rascalml1-0.13.5/assets/jars/rascal.jar]
  - [mvn://org.rascalml1--typepal--0.8.6]
  - [mvn://junit--junit--4.13.1]
  - [mvn://org.hamcrest--hamcrest-core--1.3]
  - [file:///C:/Users/Roni/.vscode/extensions/usethesource.rascalml1-0.13.5/assets/jars/rascal-lsp.jar]
rascal>import RunnerJson;
ok
rascal>RunnerJson::main(["instance/spec_types.v1"]);
{"success":true,"module":"Verilang","parseOk":true,"typeCheckOk":true,"semanticOk":true,"typeErrors":[],"semanticErrors":[],"output":["defmodule TypeExamples\n\nndefspace Person endndefspace Group : Person endndefvar age : int otherAge : int active : bool verified : bool score : real otherScore : real name : string\notherName : string initial : char p : Person q : Person g : Group endndefoperator samePerson : Person -> Person -> bool endndefoperator belongsTo : Person -> Group -> bool endndefoperator combinesScores : real -> real -> real endndefexpression age = otherAge endndefexpression active and verified\nendndefexpression score = otherScore endndefexpression name = otherName endndefexpression initial = otherInitial endndefexpression p = q endndefexpression p belongsTo g endndefexpression p samePerson q endndefexpression (forall member in Group . member belongsTo g) endnend"],"error":"","codigoFormateado":"defmodule TypeExamples\n\nndefspace Person endndefspace Group : Person endndefvar age : int otherAge : int active : bool verified : bool score : real otherScore : real name : string otherName : string initial : char p : Person q : Person g : Group endndefoperator samePerson : Person -> Person -> bool\nendndefoperator belongsTo : Person -> Group -> bool endndefoperator combinesScores : real -> real -> real endndefexpression age = otherAge endndefexpression\nactive and verified endndefexpression score = otherScore endndefexpression name = otherName endndefexpression initial = otherInitial endndefexpression p =\nq endndefexpression p belongsTo g endndefexpression p samePerson q endndefexpression (forall member in Group . member belongsTo g) endnend","resumen":"Programa Verilang parseado, chequeado y generado correctamente."}]
ok
rascal>
```

6. Modulo RunnerJson.rsc

```
import RunnerJson;
RunnerJson::main(["instance/spec_types.v1"]);

// Resultado representativo:
{
  "success": true,
  "module": "VeriLang",
  "parseOk": true,
  "typeCheckOk": true,
  "semanticOk": true,
  "typeErrors": [],
  "semanticErrors": [],
  "output": ["..."],
  "error": "",
  "codigoFormateado": "...",
}
```

```
RunnerJson::main(["instance/errors/spec_unknown_space_error.v1"]);

// Resultado representativo:
{
  "success": false,
  "parseOk": true,
  "typeCheckOk": false,
  "semanticOk": false,
  "typeErrors": ["error("Undefined space `Person`, ...)"]
}
```

```
rascal>NumberJson:=main(["instance/spec_types.v1"]);
{"success":true,"module":"Verilang","parseOk":true,"typeCheckOk":true,"semanticOk":true,"typeErrors":[],"semanticErrors":[],"output":["defmodule TypeExamples\n\n\\n\\ndefspace Person
end\\ndefspace Group : Person end\\ndefvar age : int otherAge : int active : bool verified : bool score : real otherScore : real name : string otherName : string initial : char ot
herInitial : char p : Person q : Person g : Group end\\ndefoperator samePerson : Person -> Person -> bool end\\ndefoperator belongsTo : Person -> Group -> bool end\\ndefoperator com
binescores : real -> real -> real end\\ndefexpression age = otherAge end\\ndefexpression active and verified end\\ndefexpression score = otherScore end\\ndefexpression name = otherNa
me end\\ndefexpression initial = otherInitial end\\ndefexpression p = q end\\ndefexpression p belongsTo g end\\ndefexpression p samePerson q end\\ndefexpression (forall member in Group .
member belongsTo g) end\\nend"],"error":"","codigoformateado":"defmodule TypeExamples\\n\\n\\ndefspace Person end\\ndefspace Group : Person end\\ndefvar age : int otherAge : int ac
tive : bool verified : bool score : real otherScore : real name : string otherName : string initial : char p : Person q : Person g : Group end\\ndefoperator samePerson : Person ->
Person -> bool end\\ndefoperator belongsTo : Person -> Group -> bool end\\ndefoperator combinescores : real -> real -> real end\\ndefexpression age = otherAge end\\ndefexpressio
n active and verified end\\ndefexpression score = otherScore end\\ndefexpression name = otherName end\\ndefexpression initial = otherInitial end\\ndefexpression p = q
end\\ndefexpression p belongsTo g end\\ndefexpression p samePerson q end\\ndefexpression (forall member in Group . member belongsTo g) end\\nend","resumen":"'Programa Verilang parsea
do, chequeado y generado correctamente.'"}
ok
```

7. Aplicacion Kotlin

Archivo Kotlin	Funcion
Main.kt	Crea la ventana principal con titulo VeriLang.
MainWindow.kt	Construye la interfaz grafica, el selector de archivos .vl y el panel de resultados.
LangService.kt	Ejecuta Rascal como subprocesso y envia comandos a RunnerJson.rsc.
RunResult.kt	Define la estructura de datos que representa el JSON recibido desde Rascal.

Pagina 13

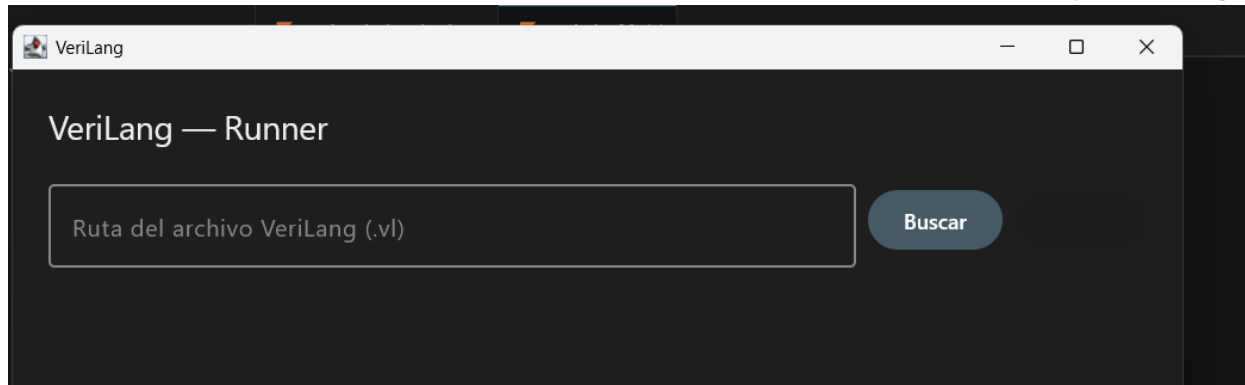


Figura 4. Interfaz final de la aplicacion Kotlin con titulo VeriLang y selector de archivos .vl.

8. Flujo de ejecucion

El flujo completo de ejecucion desde Kotlin es el siguiente:

Paso	Descripcion
1	El usuario selecciona un archivo VeriLang .vl desde la interfaz.
2	MainWindow.kt llama a LangService.run con la ruta del archivo.
3	LangService.kt abre rascal-shell-stable.jar como proceso Java.
4	El servicio escribe en stdin los comandos import RunnerJson; y RunnerJson::main([...]).
5	RunnerJson.rsc ejecuta parseMainModule, modulesTModelFromTree y generator.
6	Rascal imprime un JSON con el estado de ejecucion y los mensajes producidos.
7	Kotlin extrae el JSON, lo decodifica como RunResult y actualiza la interfaz.

Este flujo conserva el comportamiento de la ejecucion por terminal, pero lo expone desde el runtime de Kotlin. Si el programa es valido, se muestra el codigo generado; si contiene errores semanticos, la interfaz presenta los mensajes correspondientes.

9. Configuracion y dependencias

El proyecto usa Maven para las dependencias Rascal y TypePal, y Gradle para construir y ejecutar la aplicacion Kotlin. La entrega incluye Gradle Wrapper para facilitar la ejecucion desde otro entorno.

```

pom.xml
org.rascalimpl:rascal
org.rascalimpl:typepal:0.8.6

kotlin-app/build.gradle.kts
kotlin("jvm") version "1.9.22"
org.jetbrains.compose version "1.6.0"
kotlinx-serialization-json
kotlinx-coroutines

```

kotlin-app/gradlew.bat

ejecuta la aplicacion sin requerir Gradle global

El archivo rascal-shell-stable.jar queda en la raiz del proyecto. LangService.kt lo localiza desde la aplicacion Kotlin y lo usa para abrir la consola Rascal en modo subprocesso.

```
PS C:\Users\Roni\proyecto_rascal> Copy-Item "$env:USERPROFILE\.vscode\extensions\usethesource.rascalimpl-0.13.5\assets\jars\rascal.jar" "C:\Users\Roni\proyecto_rascal\rascal-shell-stable.jar"
PS C:\Users\Roni\proyecto_rascal> dir C:\Users\Roni\proyecto_rascal\rascal-shell-stable.jar

Directory: C:\Users\Roni\proyecto_rascal

Mode                LastWriteTime         Length Name
----                -
-a----           22/04/2026      3:22      82162555 rascal-shell-stable.jar
```

Figura 5. Archivo rascal-shell-stable.jar ubicado en la raiz del proyecto despues de copiar rascal.jar.

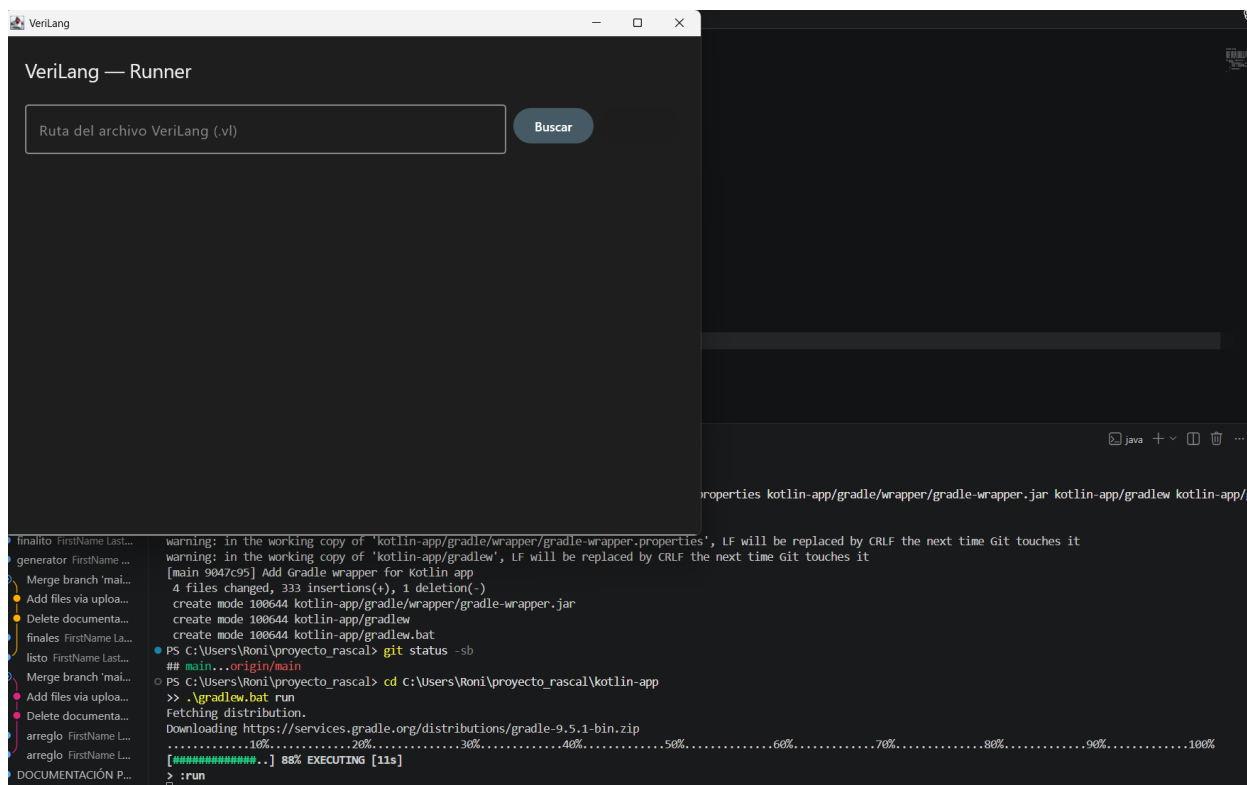


Figura 6. Ejecucion de la aplicacion con Gradle Wrapper mediante .\gradlew.bat run.

10. Casos de prueba y evidencias

La solucion se valido con pruebas de ejecucion directa en Rascal, pruebas del puente JSON y pruebas desde la interfaz Kotlin. Los casos seleccionados demuestran tanto ejecuciones correctas como reportes de error.

Prueba	Entrada	Resultado observado
Rascal valido	main() con instance/spec_types.vl	Parser OK, checker OK, generacion OK, int: 0.
Rascal invalido	main(cwd:///instance/errors/spec_unknown_space_error.vl)	Error Undefined space `Person`, no genera codigo, int: 1.
RunnerJson valido	RunnerJson::main(["instance/	JSON con success=true, parseOk=true,

	spec_types.vl"]])	typeCheckOk=true, semanticOk=true.
RunnerJson invalido	RunnerJson::main(["instance/errors/spec_unknown_space_error.vl"])	JSON con success=false, typeCheckOk=false, semanticOk=false y Undefined space `Person`.
Kotlin valido	Seleccionar instance/spec_types.vl en la UI	Parse OK, Types OK, Semantica OK y salida generada.
Kotlin invalido	Seleccionar instance/errors/spec_unknown_space_error.vl en la UI	Parse OK, Types FAIL, Semantica FAIL y error Undefined space `Person`.

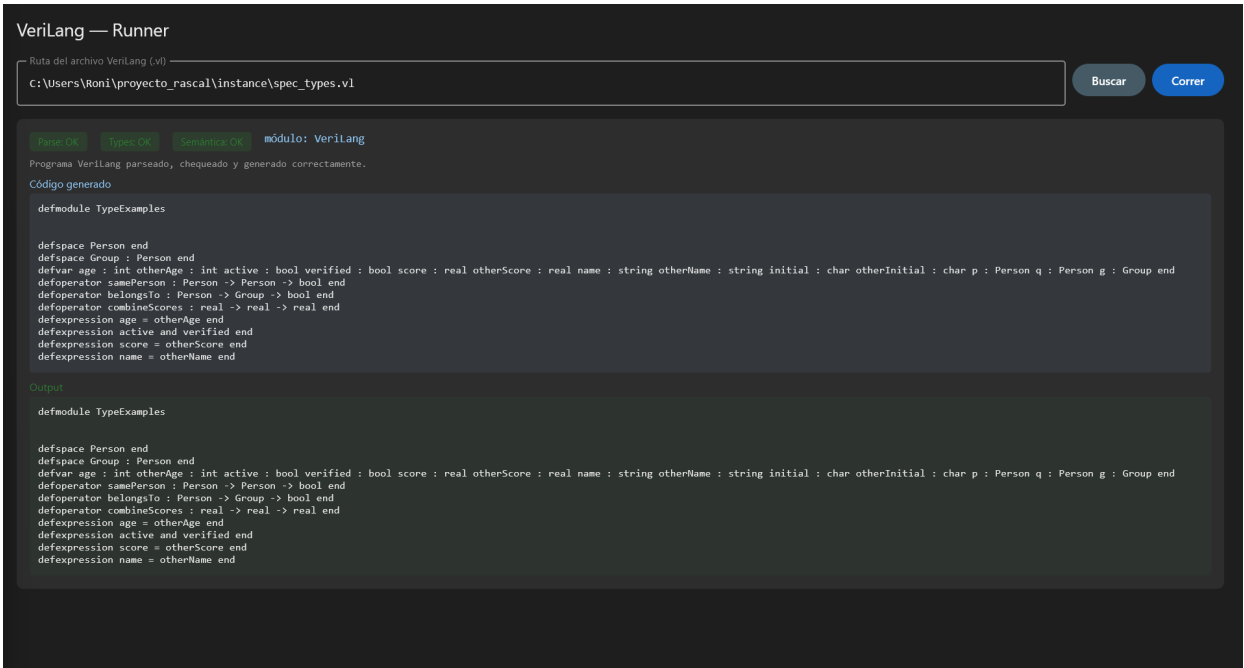


Figura 7. Ejecucion final valida desde la interfaz VeriLang, mostrando codigo generado y output.

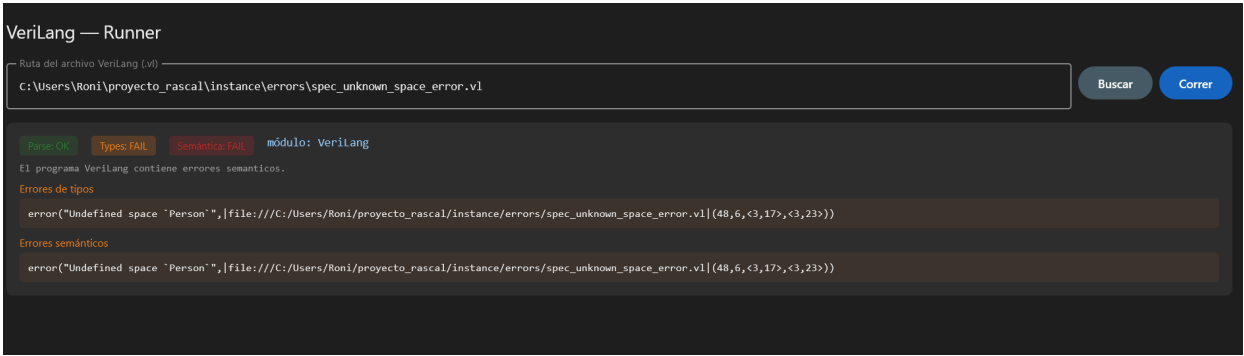


Figura 8. Ejecucion final invalida desde la interfaz VeriLang, mostrando Types FAIL y Semantica FAIL.

11. Cobertura de criterios funcionales del lenguaje

La implementacion final de VeriLang mantiene y expone desde Kotlin las capacidades semanticas centrales del lenguaje. En particular, la solucion cubre las anotaciones de tipo, la correspondencia de tipos y la existencia de elementos en estructuras de datos tipadas. Estos criterios se verifican tanto con archivos fuente .vl como con ejecuciones desde Rascal, RunnerJson y la interfaz Kotlin.

Criterio funcional	Implementacion final	Evidencia dentro del proyecto
Anotaciones de tipo y tipos definidos por usuario	La gramatica acepta tipos primitivos int, bool, real, string y char, y tambien dominios definidos por usuario mediante identificadores. Las estructuras pueden declararse como defspace Nombre : Tipo end.	Syntax.rsc define Domain y Space; AST.rsc conserva typedSpace; Generator.rsc vuelve a generar defspace Group : Person end; instance/spec_types.vl contiene Person, Group, Set y Element.
Correspondencia de tipos	Checker.rsc compara los tipos declarados contra los tipos usados en expresiones, operadores infijos, reglas, invocaciones, logica booleana y cuantificadores. Las firmas de operadores se modelan como functionType y se validan contra los argumentos.	instance/spec_types.vl ejecuta operadores samePerson y belongsTo correctamente; instance/errors/spec_typed_space_quantifier_type_error.vl y spec_type_error.vl demuestran errores de tipos controlados.
Existencia de elementos en estructuras de datos	Cuando una estructura tipada usa un tipo de usuario, el checker registra el uso contra el rol spaceld. Si el tipo no existe como defspace, TypePal reporta un espacio indefinido.	instance/errors/spec_unknown_space_error.vl y spec_typed_space_unknown_element_error.vl producen Undefined space; la interfaz Kotlin muestra Types FAIL y Semantica FAIL para el caso invalido.
Tipos dentro de definiciones anidadas	Los cuantificadores sobre estructuras tipadas usan el tipo de elemento de la estructura. Por ejemplo, si Group : Person, la variable ligada en forall member in Group toma tipo Person.	instance/spec_typed_space_quantifier_good.vl y instance/spec_types.vl validan cuantificadores correctos; la salida de Kotlin para spec_types.vl muestra Parse OK, Types OK y Semantica OK.

La ejecucion grafica conserva estos resultados: un programa valido muestra los tres estados en OK y un programa con uso de tipo no declarado conserva el parseo, pero marca fallos en tipos y semantica. De esta forma, Kotlin no sustituye el checker de Rascal; lo invoca y presenta su resultado de manera visible.

The screenshot shows the VeriLang Runner interface. At the top, the file path is C:\Users\Roni\proyecto_rascal\instance\spec_types.vl. Below the path, there are three status indicators: Parse OK, Types OK, and Semantica OK, all in green. The text below states: "Programa VeriLang parseado, chequeado y generado correctamente." The "Código generado" section shows the generated VeriLang code, which includes a module definition and several type and function declarations. The "Output" section shows the same generated code, confirming the successful execution.

```

VeriLang — Runner
Ruta del archivo VeriLang (.vl)
C:\Users\Roni\proyecto_rascal\instance\spec_types.vl
[Parse OK] [Types OK] [Semantica OK] módulo: VeriLang
Programa VeriLang parseado, chequeado y generado correctamente.
Código generado
defmodule TypeExamples
  defspace Person end
  defspace Group : Person end
  defvar age : int otherAge : int active : bool verified : bool score : real otherScore : real name : string otherName : string initial : char otherInitial : char p : Person q : Person g : Group end
  defoperator samePerson : Person -> Person -> bool end
  defoperator belongsTo : Person -> Group -> bool end
  defoperator combineScores : real -> real -> real end
  defexpression age = otherAge end
  defexpression active and verified end
  defexpression score = otherScore end
  defexpression name = otherName end
Output
defmodule TypeExamples
  defspace Person end
  defspace Group : Person end
  defvar age : int otherAge : int active : bool verified : bool score : real otherScore : real name : string otherName : string initial : char otherInitial : char p : Person q : Person g : Group end
  defoperator samePerson : Person -> Person -> bool end
  defoperator belongsTo : Person -> Group -> bool end
  defoperator combineScores : real -> real -> real end
  defexpression age = otherAge end
  defexpression active and verified end
  defexpression score = otherScore end
  defexpression name = otherName end

```

Figura 9. Ejecucion valida que demuestra anotaciones de tipo, tipos definidos por usuario y cuantificador sobre estructura tipada.

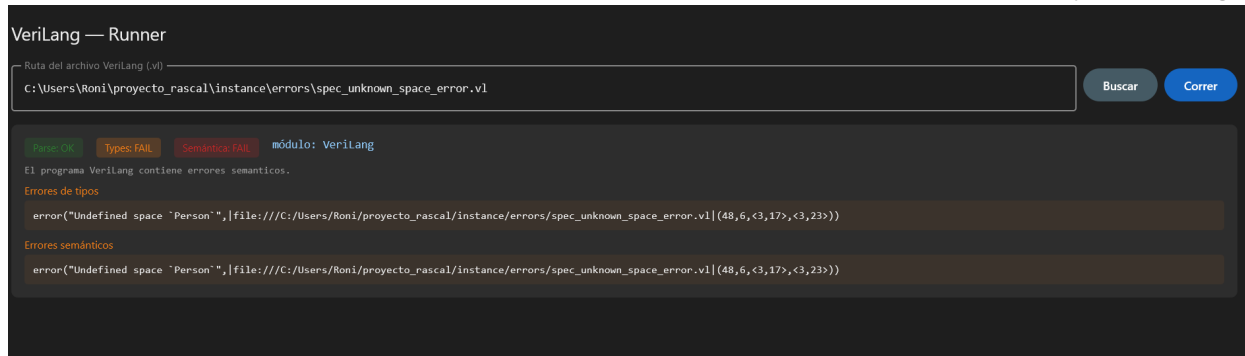


Figura 10. Ejecucion invalida que demuestra la regla de existencia para un tipo de usuario no declarado.

12. Como ejecutar el proyecto

Para ejecutar la parte Rascal, se abre una terminal Rascal ubicada en la raiz del proyecto y se importan los modulos correspondientes.

```
import Main;
main();
main(| cwd:///instance/errors/spec_unknown_space_error.vl |);

import RunnerJson;
RunnerJson::main(["instance/spec_types.vl"]);
RunnerJson::main(["instance/errors/spec_unknown_space_error.vl"]);
```

Para ejecutar la aplicacion Kotlin, se usa el Gradle Wrapper incluido en la carpeta kotlin-app.

```
cd kotlin-app
.\gradlew.bat run
```

Una vez abierta la ventana, se selecciona un archivo .vl. Para un programa valido, la interfaz presenta estados OK y salida generada. Para un programa invalido, presenta el error semantico devuelto por TypePal.

13. Contenido de entrega

Elemento	Descripcion
Proyecto Rascal	Modulos de VeriLang en src/main/rascal.
Aplicacion Kotlin	kotlin-app con Compose Desktop, Gradle Wrapper y código fuente.
RunnerJson.rsc	Modulo puente entre Rascal y Kotlin.
rascal-shell-stable.jar	Jar usado por la aplicacion para ejecutar Rascal.
Casos de prueba	Archivos .vl validos e invalidos dentro de instance/.
Documento final	Explicacion de arquitectura, ejecucion y evidencias.

Los directorios generados por ejecucion o compilacion, como target/, build/, bin/ y .gradle/, no forman parte del contenido fuente de la solucion.

14. Conclusiones

La version final del Proyecto 4 ejecuta VeriLang desde el entorno de Kotlin manteniendo la funcionalidad del Proyecto 3. El parser, el checker con TypePal y el generador permanecen implementados en Rascal, mientras que Kotlin proporciona una interfaz de ejecucion y visualizacion de resultados.

El modulo RunnerJson.rsc establece una frontera clara entre ambos entornos. Rascal produce un resultado JSON y Kotlin lo interpreta para mostrar estados, errores y salida generada. Esta integracion permite comprobar programas correctos y programas con errores semanticos desde una aplicacion de escritorio.

Las evidencias incluidas muestran que la solucion parsea, chequea y genera correctamente programas validos, y que reporta errores semanticos esperados en programas invalidos. La entrega incluye tambien Gradle Wrapper y el jar de ejecucion Rascal, de forma que la aplicacion Kotlin pueda ejecutarse desde la estructura final del proyecto.